

DEGGENDORF INSTITUTE OF TECHNOLOGY

SCHOOL OF APPLIED COMPUTER SCIENCE

Software Architecture

©Arc42 inspired [1]

Authors:
Jay Ravani

Last Modifications: July 17, 2026

Contents

1	Introduction & Goals	2
2	Constraints	3
3	Context & Scope	4
4	Solution Strategy	5
5	Building Block View	6
6	Runtime View	7
6.1	Step 1: preflight (OPTIONS)	7
6.2	Step 2: the real request	7
6.3	Step 3: rejected request	7
6.4	Why this matters	8
7	Deployment View	9
7.1	The three containers	9
7.2	Ports	9
7.3	First-run data load	10
7.4	No host port on app or database	10
7.5	Startup order	10
7.6	Certificate trust across recreation	10
7.7	Network	10
8	Crosscutting Concepts	12
8.1	TLS termination	12
8.2	Trust follows the CA, not the certificate	12
8.3	The API-key gate	12
8.4	CORS: two concerns kept apart	13
8.5	Open items	13
9	Architectural Decisions	14
9.1	ADR-001: One reverse proxy per service, not a shared gateway . . .	14
9.2	ADR-002: Proxy and app in separate containers	14

9.3	ADR-003: Reuse one CA across container recreation	15
9.4	ADR-004: Static API key as a prototype, not production auth	15
9.5	ADR-005: Group containers into per-concern namespaces, named - .	15
10	Quality Requirements	17
11	Risks & Technical Debt	18
12	Glossary	19

Chapter 1

Introduction & Goals

Chapter 2

Constraints

Table 2.1: External and organizational constrains.

Nr.	Constraint
01	-

Chapter 3

Context & Scope

Chapter 4

Solution Strategy

Chapter 5

Building Block View

Chapter 6

Runtime View

A browser client calling ignis through the proxy produces two requests: a CORS preflight, then the real request. This section traces both, plus a rejected request.

6.1 Step 1: preflight (OPTIONS)

Building Configurator calls `GET /api/v1/fields` from a page on `http://localhost:5173`. The target is a different origin (`https://localhost`) and the request carries a custom header (`X-API-Key`), so the browser first sends an automatic `OPTIONS` preflight to ask permission. A preflight never carries the custom header itself.

Caddy matches the preflight on method alone, before the API-key check, and answers it: it echoes the allowed origin, lists the allowed methods and headers, and returns `204 No Content`. The preflight never reaches ignis.

6.2 Step 2: the real request

The browser now sends `GET /api/v1/fields` with `X-API-Key` attached. Caddy checks the key. On a match, it forwards the request to `ignis-app:8080` over the internal Docker network as plain HTTP (encryption already ended at Caddy). ignis handles it as if no proxy existed: it checks the `Origin` against `ALLOWED_ORIGINS` and adds `Access-Control-Allow-Origin` to the response. Caddy passes the response back unchanged.

6.3 Step 3: rejected request

A request with a missing or wrong `X-API-Key` hits Caddy's fallback rule and gets `403 Forbidden`. It never reaches ignis. ignis publishes no host port, so the proxy is the only way in.

6.4 Why this matters

ignis behaves identically with or without the proxy in front of it. Caddy adds the TLS, the key check, and the preflight response; ignis's code knows nothing about any of it.

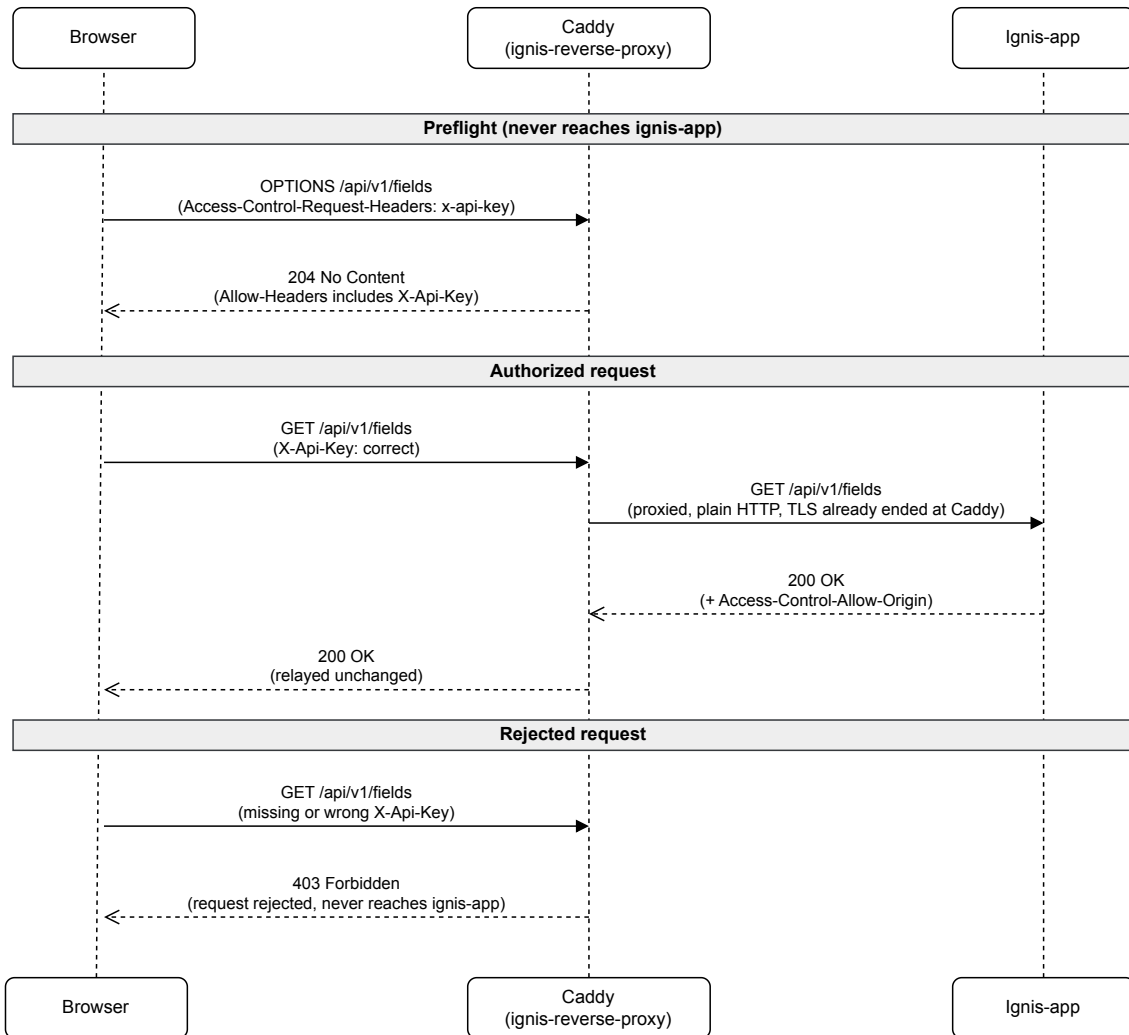


Figure 6.1: Runtime View: sequence of a preflight, an authorized request, and a rejected request. ignis's request handling is identical whether or not Caddy is in front of it; only Caddy's own behaviour differs across these three cases.

Chapter 7

Deployment View

7.1 The three containers

ignis runs as three containers, defined in `environment/docker-compose.yml`. They form the `building-simulation` namespace (the Docker Compose project name), shared with other building-modelling services such as `buem`. The stack needs nothing on the host except Docker.

1. **ignis-reverse-proxy**: Caddy, from the `caddy:2.11-alpine` image. The only container that publishes a host port. Every request passes through it first.
2. **ignis-app**: the `bin/app` binary, built from `environment/Dockerfile`. Listens on the internal port (default 8080), publishes no host port, and reaches the database at `ignis-db`.
3. **ignis-db**: PostgreSQL, from `postgres:17-alpine`. Publishes no host port. Its data lives in a named volume (`ignis-db-data`) that survives `docker compose down/up`, but not `down -v`.

Every component is a container, so the whole stack can be built into images, pushed to a registry, and run elsewhere with the same compose file. There is no host-installed database to set up separately.

7.2 Ports

Two ports, with different rules.

- **Internal port** (`APP_PORT`, default 8080): the port `ignis-app` listens on inside its container. Container isolation means it never clashes with other services, so it stays the same everywhere. It is set once in `.env` and passed to the app, its health check, and the proxy's upstream, rather than hardcoded in each.

- **Host port** (`HOST_HTTPS_PORT`, default 443): the port `ignis-reverse-proxy` publishes. This is the only port that can clash: two services cannot both take host 443. An orchestration layer assigns a free port here per service.

7.3 First-run data load

A fresh `ignis-db` volume is empty. Load the TABULA data once, after first start: `docker compose exec ignis-app ./bin/build_db`. The data persists afterward. `build_db` drops and recreates all tables, so it is a manual step, not part of startup.

7.4 No host port on app or database

`ignis-app` and `ignis-db` declare no ports:, so nothing outside the Docker network can connect to them. `ignis-app` is reachable only from `ignis-reverse-proxy`, and `ignis-db` only from `ignis-app`, each by its service name. The proxy is not an add-on in front of an open service: it is the only way in.

7.5 Startup order

`depends_on:` `condition:` `service_healthy` chains startup: `ignis-db` must accept connections (checked with `pg_isready`) before `ignis-app` starts, and `ignis-app` must be healthy before `ignis-reverse-proxy` starts. The app's health check makes a real `GET /health` call (not `HEAD`, which the router does not register). Nothing starts serving before what it depends on is ready.

7.6 Certificate trust across recreation

A fresh Caddy container would generate a new, untrusted certificate authority, breaking any trust the browser already had. `ignis-reverse-proxy` avoids this by mounting the host's Caddy data directory (`~/local/share/caddy`, set via `CADDY_DATA_DIR`) into the container, so it reuses the same CA. A browser that ran `caddy trust` once keeps trusting it.

This is a local-development convenience, tied to one machine. A real deployment replaces it with either public HTTPS (Caddy's built-in Let's Encrypt) or a shared internal CA every client trusts.

7.7 Network

All three containers share one Docker Compose network (`building-simulation_default`). Docker's DNS resolves each service name to its container: the proxy reaches the app

at `ignis-app`, the app reaches the database at `ignis-db`. This only works within the same network, which is why keeping everything on it (and nothing on a host port except the proxy) keeps the stack self-contained and closed.

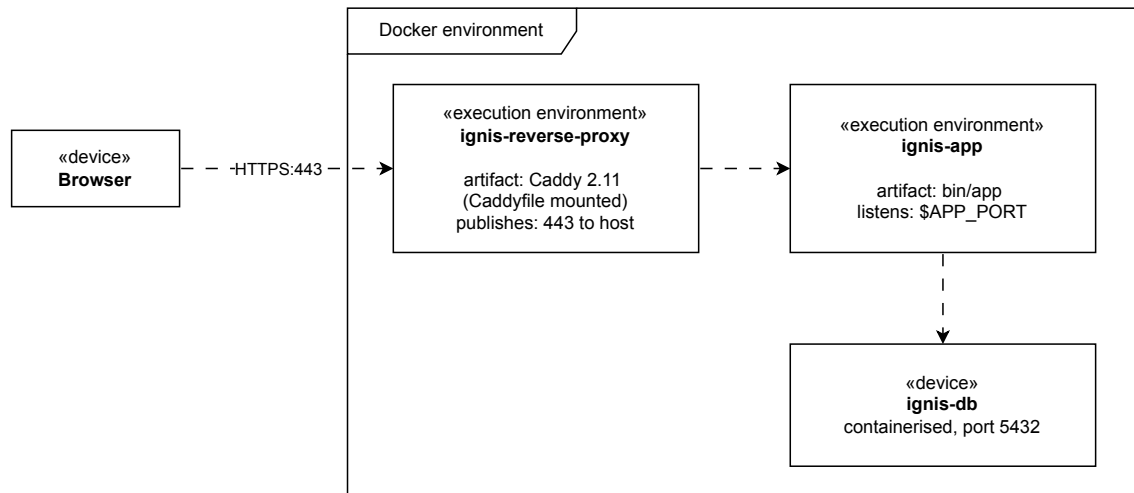


Figure 7.1: Deployment View: the three containers, all on one Docker Compose network in the building-simulation namespace. Only `ignis-reverse-proxy` publishes a host port; `ignis-app` and `ignis-db` are reachable only by service name from within the network. The stack needs nothing on the host except Docker.

Chapter 8

Crosscutting Concepts

8.1 TLS termination

ignis's code has no TLS, no certificates, no HTTPS server. Encryption ends at the proxy: it holds the certificate, decrypts the request, and forwards it as plain HTTP over the internal Docker network. Any service behind the proxy gets this for free: certificates live in one place, not in every service.

The internal hop from proxy to app is plain HTTP, so it is only as safe as the network it crosses. That is fine while the network is one we control (a single Docker network). It stops being fine if that hop ever crosses infrastructure we don't control, which is why mutual TLS on that hop is listed as future work.

8.2 Trust follows the CA, not the certificate

A client doesn't trust a certificate on its own. It trusts the certificate authority (CA) that signed it, and that trust then covers every certificate the same CA signs. This is why per-service proxies scale: sign each one's certificate with the same CA (a public CA for internet-facing services, a shared internal CA for service-to-service), and a client establishes trust once, not once per service.

Locally this is proven directly: `caddy trust` trusts the local CA once, and every certificate it signs afterward is trusted, including from a freshly recreated container.

8.3 The API-key gate

Caddy checks for a valid `X-API-Key` before forwarding to `ignis`. `ignis` knows nothing about this key: it never reads it.

This is a prototype stand-in, not production auth. The current caller, Building Configurator, runs in the browser, so any key it sends is visible in its page source to anyone with developer tools. A static key shipped to the browser stops casual access but keeps no real secret. It stands in for the real caller in the target architecture,

an orchestration layer running as a backend service, which can hold a credential the client never sees. What replaces the static key is settled in ADR-4.

8.4 CORS: two concerns kept apart

A cross-origin request with a custom header triggers a preflight OPTIONS before the real request. ignis's own CORS middleware only knows Content-Type, correctly, since X-API-Key is not part of ignis's API; it exists only because of the proxy. Teaching ignis about it would tie ignis's code to its deployment.

So Caddy answers the preflight itself, advertising the headers and origins it accepts. The real request still passes ignis's own origin check (ALLOWED_ORIGINS) when it arrives. Each layer handles its own concern.

8.5 Open items

- Mutual TLS on the proxy-to-app hop, for deployments where that hop leaves infrastructure we control.
- Replacing the static API key with a credential issued by the real orchestration layer, once it exists.

Chapter 9

Architectural Decisions

9.1 ADR-001: One reverse proxy per service, not a shared gateway

Status: Decided

Context: ignis needs TLS and an access check in front of it. Other services (buem and more) will need the same.

Decision: Each service gets its own proxy, deployed with it. No shared gateway.

Reason: A shared gateway ties every service's uptime to one process. Reconfiguring the proxy for one service (as we did to fix its CORS handling) would risk every other service behind it. A per-service proxy limits that to one service, and the same Caddy setup drops straight onto the next service.

Rejected: A single shared API gateway. Reasonable at a different scale, but the wrong fit for per-service ownership.

9.2 ADR-002: Proxy and app in separate containers

Status: Decided

Context: Caddy and the app could run as two processes in one container.

Decision: They run as separate containers.

Reason: Docker expects one process per container. Two processes under a wrapper script lose supervision: if the app crashed, Docker wouldn't notice, since the script keeps running. Separate containers also let the proxy be reloaded or recreated without touching the app, which we relied on repeatedly during this work.

Rejected: One container with a process supervisor (e.g. supervisord). Extra complexity for nothing this deployment needs.

9.3 ADR-003: Reuse one CA across container recreation

Status: Decided for local development; production open

Context: A fresh Caddy container generates a new, untrusted CA, breaking trust the browser already had.

Decision: Mount the host's Caddy CA storage into the container so it reuses the same CA.

Reason: Trust attaches to the CA, not the individual certificate (see Crosscutting Concepts). Reuse one CA and trust survives recreation, reloads, and adding more services signed by the same CA.

Rejected: Re-running caddy trust after every recreation. Defeats the point and doesn't scale past one machine.

Open for production: Mounting one developer's local CA is local-only. Production needs public HTTPS (Let's Encrypt) or a shared internal CA. Which one depends on where ignis sits relative to the internet, not yet decided.

9.4 ADR-004: Static API key as a prototype, not production auth

Status: Open

Context: ignis should be reachable only by a trusted caller. The real caller, an orchestration layer, doesn't exist yet. Building Configurator stands in for it.

Decision (provisional): Caddy gates on a static, shared X-API-Key.

Reason: Enough to prove the proxy can enforce "only a known caller gets in." Not enough for production: Building Configurator runs in the browser, so the key is visible in its page source.

Rejected for now, not yet chosen between: a short-lived token from the real orchestration layer, or mutual TLS giving each service its own identity. Both wait until that layer exists. Revisit then.

9.5 ADR-005: Group containers into per-concern namespaces, named -

Status: Decided

Context: The energy-modelling system is several services (ignis, buem, and more), each with a proxy, app, and sometimes a database.

Decision: Group containers by concern into a Docker Compose namespace (project name), here building-simulation, shared by all building-modelling services. Name

each container `<service>-<role>`: `ignis-app`, `ignis-db`, `ignis-reverse-proxy`, and later `buem-app`, `buem-reverse-proxy`, and so on.

Reason: A container name then says both which service it belongs to and what role it plays, and the namespace groups related services together. The host port is the only thing that can clash between them, so it is set per service via `HOST_HTTPS_PORT`; the internal port stays fixed, since container isolation keeps it from clashing.

Rejected: Directory-derived project names and ad-hoc container names. They don't convey role or concern and don't group cleanly as more services are added.

Open: When `buem` joins, two repos each declaring the same namespace will coexist but `Compose` may warn about "orphan" containers. A single top-level `compose (include:)` or a shared external network resolves it. Decide when `buem` is wired in.

Chapter 10

Quality Requirements

Chapter 11

Risks & Technical Debt

Chapter 12

Glossary

Bibliography

- [1] D. P. Hruschka and D. G. Starke. Arc42. URL <http://www.arc42.org>. pages